

Weather App

Project

CT30A2910 Introduction to Web Programming

Lappeenranta-Lahti University of Technology

Summer 2025

Lilla Bagossi

Features and functionalities

Below you may read about the features of the application in detail.

The user can search for any city in the world using the search bar and pressing the “Search” button. The first panel on the app will display the current weather (with relevant icon) at the given location including city name, country code, current temperature, description of weather (e.g. “clear sky”), “feels-like” temperature, humidity, pressure, visibility, wind degree and wind speed (km/h), and sunrise and sunset time that day. All data in this panel is fetched from the OpenWeather API.

The second panel displays a 24-hour forecast for the search location using a Frappe chart. If the current time at the searched location is 14:29, the chart displays data from 15 o’clock that day till 14 next day. The current time is also the current time at the searched location, not the current UTC or UTC+3 time. There is a multi-select menu for displaying up to 3 different weather temperature predictions. The dropdown menu with “Temperature” is supposed to give options to see precipitation, humidity, and other predictions in the graph, but this hasn’t been implemented, so this menu is disabled for now, similarly to the range input at the top, which was supposed to enable the user to select the number of forecast hours.

The third panel shows a 7-day daily forecast. The selector menu there can be used to select data provider. The data varies from APIs. E.g. OpenMeteo, unlike the other 2 APIs, only had hourly forecast values, so then min, max and average daily values were calculated from that. All the weather descriptions are from OpenWeather, since the other 2 APIs didn’t provide that. Relevant icons are displayed here too based on the weather.

Next, the map by default shows temperature data, with the current city in focus, using the layer menu at the top right corner, the user can select the precipitation, pressure, or wind map.

The user can tag the currently searched location as favorite. By default, all search values are added to the “Last visited” menu at the top. By clicking on any, the search for that area is executed. The user can also use their own location (“Use current location”) to search.

The default units are Celsius, but the user can change to Fahrenheit or Kelvin also, then all temperature data on the page will be refreshed and displayed of the chosen unit. For OpenWeather API data, that means fetching data with a unit filter, for the other two APIs, the program converts the fetched data from Celsius (due to the API data configuration).

The relevant themes and weather icons are chosen based on (1) description of the weather, (2) temperature (hot if >35C), and whether it is day or night (calculated based on sunrise, sunset and current time values). There are at least 6 different themes in use currently: default, sunny, sunny-cloudy, cloudy, hot-cloudy / hot-rainy, sunny-rainy, rainy-cloudy / stormy, snowy and night. Most

have custom background; button, selection menu and input background and border colors. Theme rules are defined in CSS. Daytime has light <div> bg and 8 custom icons, and nighttime with dark bg, some extra box-shadow and 6 custom nighttime icons.

The app is responsive; there is a default layout for phones. On >middle screens the layout and content sizes are different. There are some cool cursor hover features for desktop. I also wanted to implement a cool feature of the user being able to drag buttons, but I didn't have time to implement that (yet).

Tools used

- HTML: backbone of the webpage, elements are defined there.
- CSS: for styling, dynamic movements (e.g. hover actions) and responsiveness.
- Bootstrap: managing layout for bigger screens.
- Leaflet: StreetView basemap and weather layers.
- Frappe: for the hourly forecast chart
- Geolocation API (Mozilla): get current location of the user.
- Flaticon (Freepik): Using weather icons (requires attribution to FreePik)
- Plain JavaScript: for everything else. Calculations, handling events, fetching data from APIs, choosing appropriate theme and icons.

Justification of anticipated score

Feature	Justification	Anticipated points
User can search for locations	Works (there is no control for cities with the same name, e.g. "Lahti" returns a city named Lahti in Russia because of OWM's own search results).	1 / 1
User can use his/her location GPS-coordinates (Geolocation API)	Implemented.	2 / 2
At least two data/forecast providers are used	Implemented for 24h and 7d forecast.	3 / 3
At least three data/forecast providers are used	Implemented for 24h and 7d forecast.	2 / 2

User sees the current weather at a specific location	With many weather details.	1 / 1
User sees the forecast for the next 24 hour, hourly based	With the correct local time.	3 / 3
User sees the forecast for the next 7 days	With relevant icons.	3 / 3
All the weather forecast elements uses icons (and numbers) for e.g. sunny and cloudy weathers	8 different day-time icons, and 6 for night. Considering hot weather too.	3 / 3
The look and feel of the application reflects the current weather (e.g. it is blueish, when it is cold; reddish, when it is hot;; dark, when it is night...)	Theme also reflects sunny, cloudy, rainy, stormy, snowy weather and a combination of these. + night	2 / 2
User sees simultaneously three forecast in a graph, e.g. there is temperature forecast for the next 24 hours and there are two lines telling how the data sources are providing (a bit) different data		3 / 3
User has the option to tag some locations as her favorites and thus access them from the favorites menu		2 / 2
Own feature: Searches are stored automatically, and user can access them from "Last visited" menu		1 / 1
User has an option to switch between celsius and fahrenheit degrees and kelvins	Implemented, despite API limitations.	2 / 2
Application is responsive and can be used on both desktop and mobile environment.	Desktop layout works on tablets too. But multiselect doesn't work on phones.	3 / 4
The application has clear directory structure and everything is organized well		2 / 2
Well written PDF report	(with recommended test searches in the GitHub readme)	3 / 3
Total:		37 / 40

User Manual

To see multiple datasets in the hourly graph at the same time, click on the options while holding down the CTRL key, then clicking "Search". This feature doesn't work on phones, unfortunately.

Declaration of AI usage

Only one AI tool was used, which is ChatGPT. Here is an exhaustive list of how ChatGPT was used:

- For finding weather API providers that give free access and match specific requirements (e.g. daily forecast for 7 days, hourly forecast for 24 hours).
- For inspiration for the color palette of the night theme (although not much of the recommendations were used)
- For debugging code (mostly errors related to async timing) and understanding the JavaScript concepts behind those errors. Hence some bug-fix code snippets (e.g. the interval for waiting before fetching is finished) have been used. Most of these code snippets have comments around them that indicate that they are from ChatGPT. (Generally, the source of code that is taken from elsewhere is marked with a comment)
- For inspiration for CSS decoration, layouts, and help on how to make my layouts work using Bootstrap
- For implementing the clickable `<p>` tag in the Favorites and Last visited menus.

There was no AI used in producing this document.

Extra: Suggestions for future improvement

- Hourly chart with different weather conditions. Also displaying all available datasets by default.
- User can set thematic preference, e.g. daily panel in the weekly forecast can appear with relevant theme based on forecasted weather condition)
- Advanced search, using coordinates
- Drop-down menu with matching values while typing the city name in the search bar
- Customize the multiselect button, so that it works well on both phones and desktops.